

Query Profile, History, and Insights

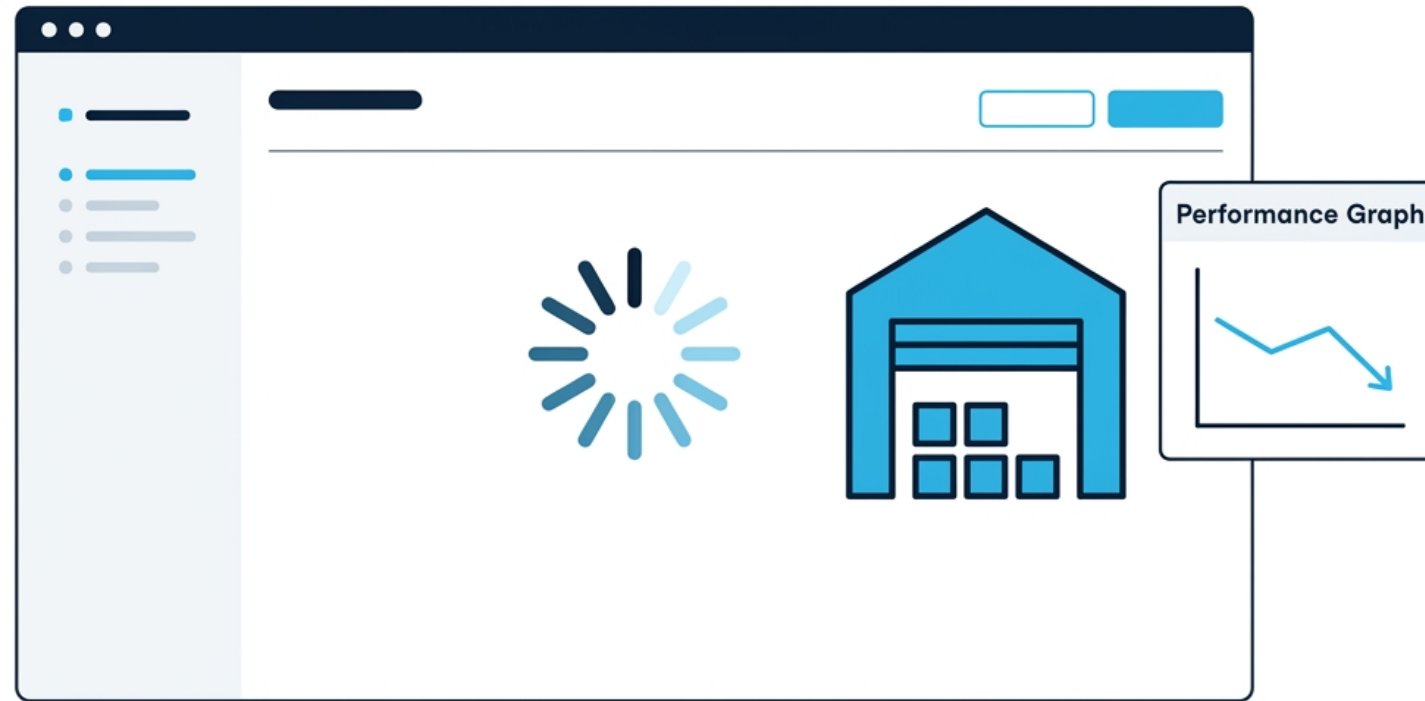
DATA PIPELINE AUTOMATION IN SNOWFLAKE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

Why Query Profile



- Snowflake might be scanning data it shouldn't touch at all

Opening the Query Profile in Snowsight

Home

Search this account, Marketplace, and documentation

Quick actions

- Upload local files**
Quickly convert data into tables.
- Load from cloud storage**
Use a SQL template to load data.
- Query data**
Create a SQL file in Workspaces.
- Invite users**
Collaborate with others on your team.

Consistent metrics & smarter insights with Semantic Views
Define business logic once and use it everywhere—across AI, BI, and SQL. Snowflake Semantic Views bring consistent, governed metrics and dimensions directly into your data platform.
[Try it out](#) [Learn more](#)

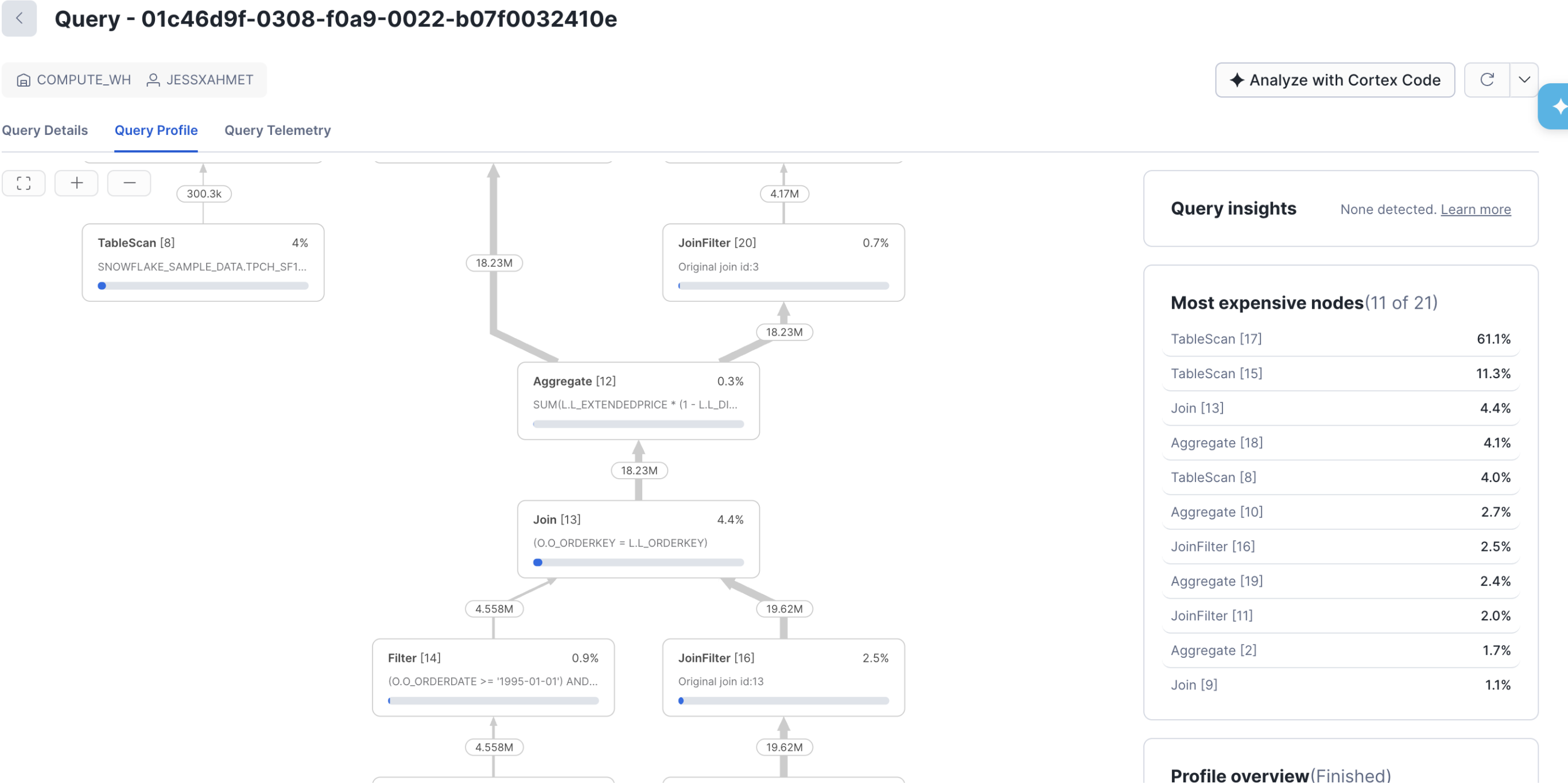
All projects

[All projects](#) [Files](#) [Notebooks](#) [Streamlit](#) [Dashboards](#) [Folders](#)

TITLE	TYPE	VIEWED ↓	UPDATED
Untitled 10.sql	SQL File	1 week ago	1 week ago
Untitled 9.sql	SQL File	1 week ago	1 week ago

Access via: Monitoring > Query History > [Select Query] > Query Profile Tab

Reading the Operator Tree



Four Common Problems Query Profile reveals

Exploding Joins

- Produces more rows than it can consume

Union without All

- Adds an Aggregate above your `UNION` to remove duplicates
- Cost = added time

Spill to Disk

- Memory-intensive operators run out of memory
- Operators such as Aggregates, Sorts or large Joins.

Inefficient Pruning

- Visible when partitions scanned close to partition total
- Filters aren't working effectively

Spill to Disk: Local vs Remote

- **Local spill** → warehouse SSD - slower than memory, but manageable
- **Remote spill** → cloud storage - much slower; must be addressed

Diagnostic Query

```
SELECT
query_id,
bytes_spilled_to_local_storage,
bytes_spilled_to_remote_storage
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE bytes_spilled_to_remote_storage > 0
ORDER BY bytes_spilled_to_remote_storage DESC
LIMIT 10;
```

QUERY_HISTORY: Account-wide Visibility

- ACCOUNT_USAGE.QUERY_HISTORY - view with 365 day retention
- INFORMATION_SCHEMA.QUERY_HISTORY() table function with 7 day retention



```
1 SELECT query_id
2   , total_elapsed_time
3   , bytes_scanned
4 FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
5 ORDER BY bytes_scanned
6 DESC LIMIT 10;
```

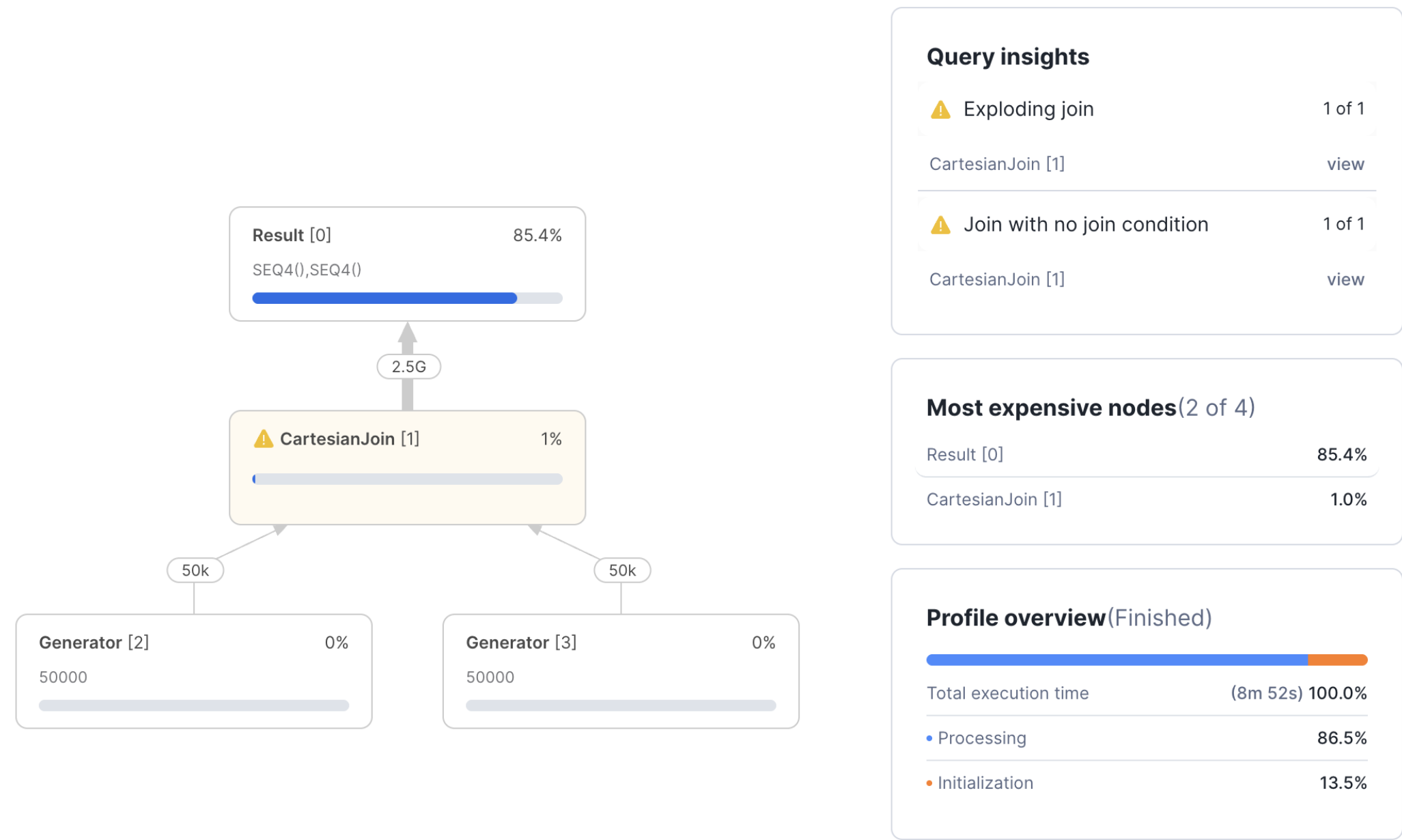
Results (2 minutes ago)

Table Chart Pivot

10 rows 20s

	QUERY_ID		# TOTAL_ELAPSED_TIME	# BYTES_SCANNED
	01c2f3cc-0108-0a6e-0022-b07f0001e026	10.0%		
	01c310ec-0208-1f1d-0022-b07f00030026	10.0%		
	+8 more		6280297437	319654449660166473147
1	01c46d9a-0308-efed-0022-b07f0031f40a		297437	60166473147
2	01c3bf4c-0308-80df-0022-b07f001dd0fa		78530	40916967480
3	01c310ec-0208-1f1d-0022-b07f00030026		69183	34418900584
4	01c2f3cc-0108-0a6e-0022-b07f0001e026		59716	33002075008
5	01c39cc1-0108-6b29-0022-b07f0019713a		24746	12650333952
6	01c46dba-0308-f0ac-0022-b07f0032313e		27751	11777518816

Query Insights: Automated Recommendations



Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

Optimization Methods

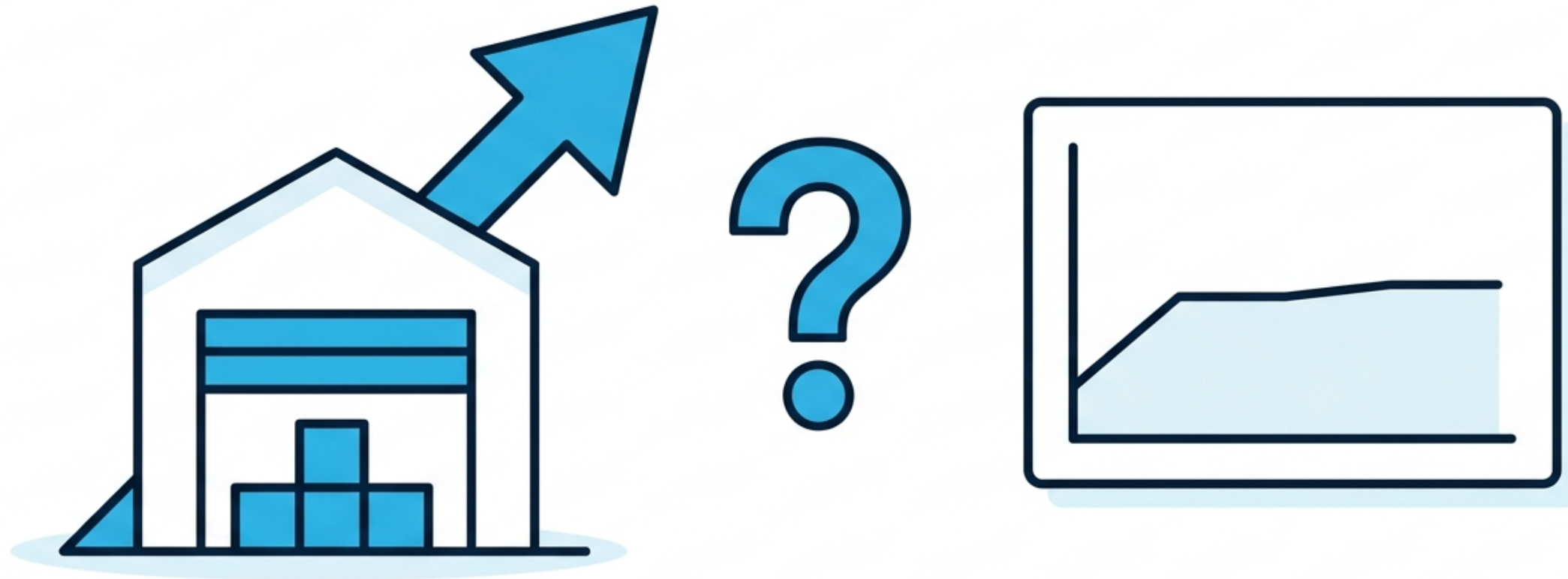
DATA PIPELINE AUTOMATION IN SNOWFLAKE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

More Compute isn't Always the Fix



Four Different Approaches



- **Search Optimization** - equality lookups on high-cardinality IDs
- **Query Acceleration Service** - unpredictable heavy scans
- **Automatic Clustering** - repeated range filters on the same columns
- **Materialized Views** - expensive aggregations run often

Search Optimization

- Helps finding queries that look for a small number of rows in a large table
- Builds a persistent access path; some micro-partitions can be skipped
- Storage + compute cost for maintaining the access path

```
ALTER TABLE logistics.shipments  
ADD SEARCH OPTIMIZATION;
```

¹ docs.snowflake.com/en/user-guide/search-optimization-service

Query Acceleration Service

- Handles large unpredictable scans
- Supports `SELECT` , `INSERT` ,
`CREATE TABLE AS SELECT` , `COPY INTO`
- `QUERY_ACCELERATION_MAX_SCALE_FACTOR` :
default 8; 0 means unlimited, not disabled

```
ALTER WAREHOUSE harbr_wh SET  
  ENABLE_QUERY_ACCELERATION = TRUE  
  QUERY_ACCELERATION_MAX_SCALE_FACTOR  
  = 8;
```

¹ docs.snowflake.com/en/user-guide/query-acceleration-service

Automatic Clustering

- For tables where the same filter columns appear in most queries
- Snowflake reorders micro-partitions so range filters can skip more data
- Background maintenance - no warehouse needed
- Trade-off: ongoing compute cost as new data arrives

```
ALTER TABLE logistics.delivery_events  
  CLUSTER BY (region, dispatch_date);
```

¹ docs.snowflake.com/en/user-guide/tables-auto-reclustering

Materialized Views

- Needed when an expensive query runs over and over again
- Pre-computes and stores a query's result; reads hit the stored result
- Snowflake maintains it automatically in the background
- SQL restrictions apply: no HAVING, limited joins, no window functions

```
CREATE MATERIALIZED VIEW
logistics.emea_summary_mv AS
  SELECT region, carrier_id,
         COUNT(*) AS shipments,
         AVG(delivery_days) AS avg_days
  FROM logistics.shipments
 WHERE region = 'EMEA'
 GROUP BY region, carrier_id;
```

¹ docs.snowflake.com/en/user-guide/views-materialized

Choosing the Right Method

Signal in the Query Profile	Method to Consider
TableScan reads almost all partitions; equality lookup on a high-cardinality column	Search Optimization
Heavy aggregation, large scans, arrives unpredictably	Query Acceleration Service
Same filter columns appear in most queries; pruning is weak	Automatic Clustering
Same expensive aggregation runs frequently on slow-changing data	Materialized View

- Methods complement each other

Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

How Caching Works in Snowflake

DATA PIPELINE AUTOMATION IN SNOWFLAKE

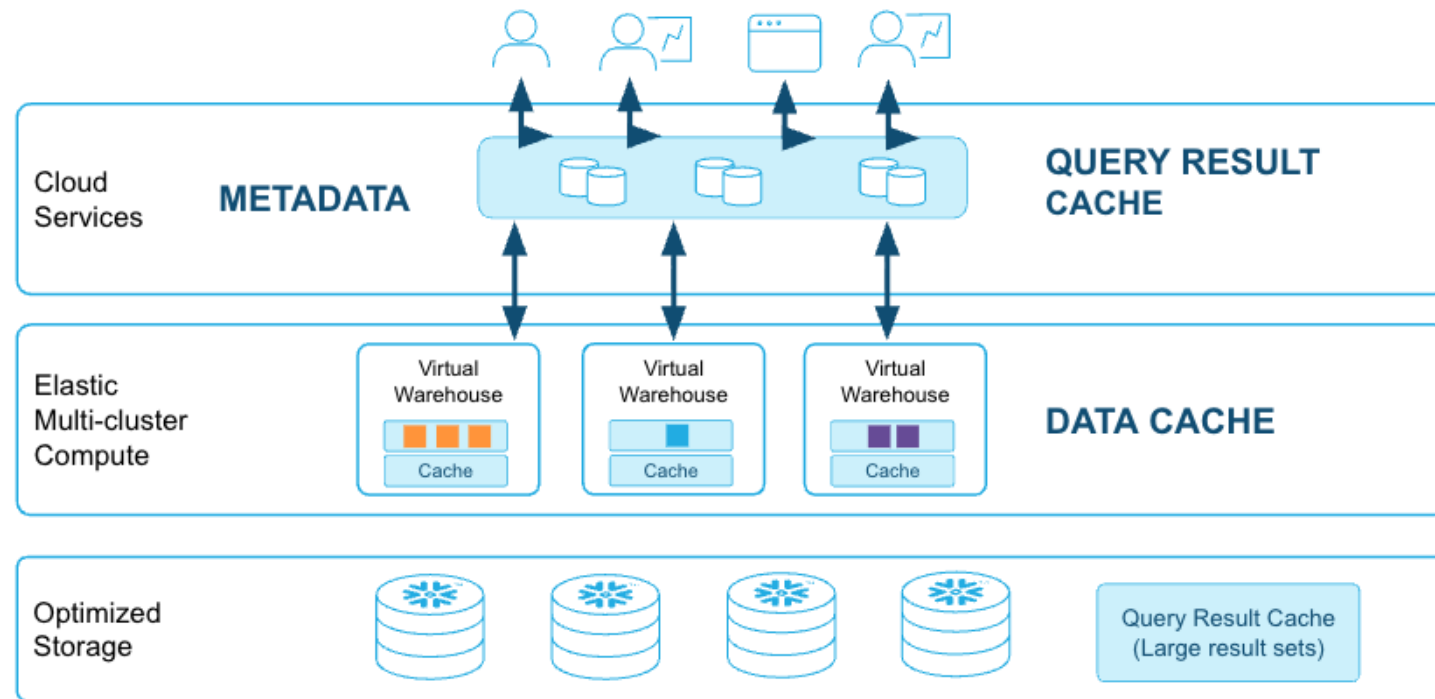


Emily Melhuish

Technical Curriculum Developer,
Snowflake

Three Caching Layers

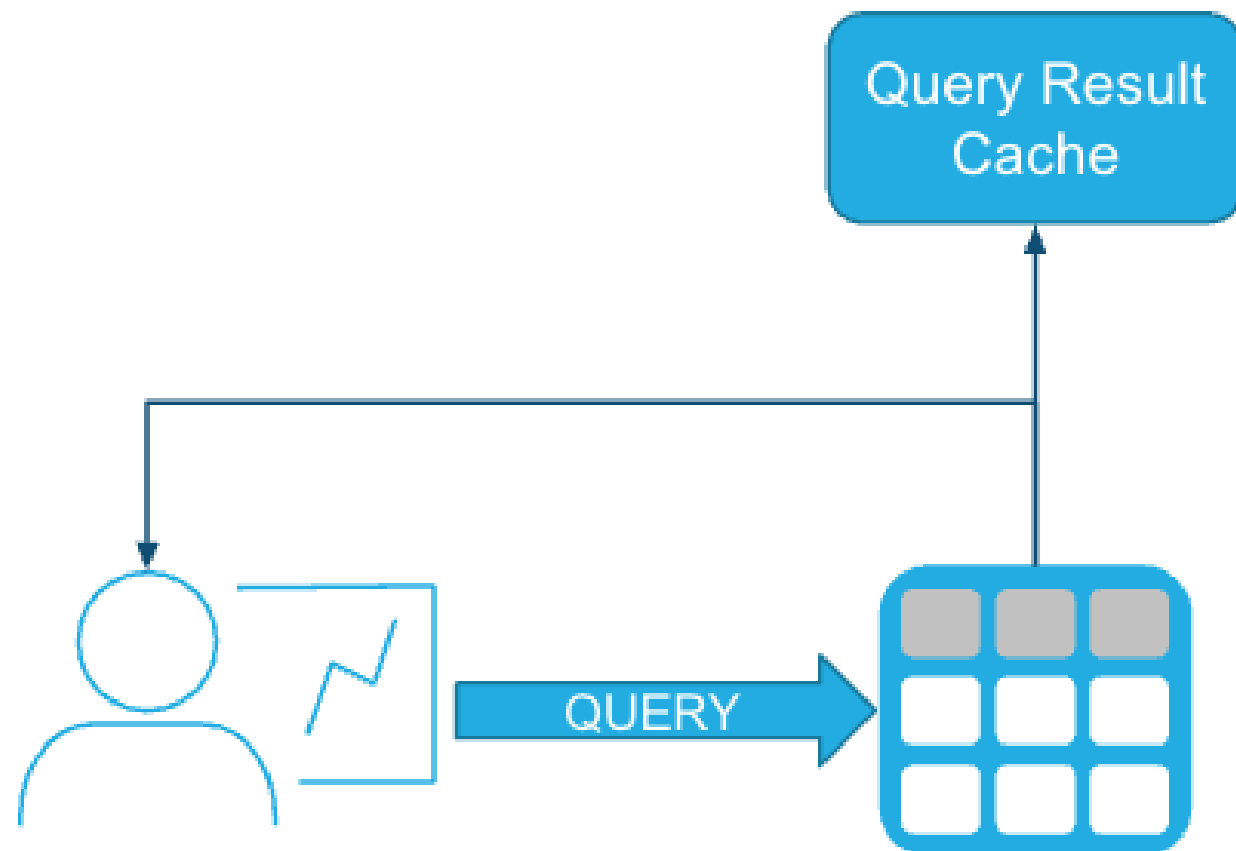
METADATA AND CACHING IN SNOWFLAKE



Snowflake caches results at **three levels** — each with different scope and duration

- **Result cache** — stores complete query results and returns them without running a warehouse
- **Warehouse or data cache** — recently scanned micro-partitions in memory and SSD
- **Metadata cache** — always on; table stats used during query planning

When the Result Cache is used (and when it isn't)



Condition	Cache Miss When...
Query text matches exactly	Casing, whitespace, or aliases differ
No non-reusable functions	functions that change between runs: RANDOM
Underlying tables unchanged	DML, reclustering, or consolidation since last run
Persisted result still available	24-hour TTL elapsed without reuse, or 31 days
Role has the required privileges	SELECT: role lacks privileges SHOW: different role

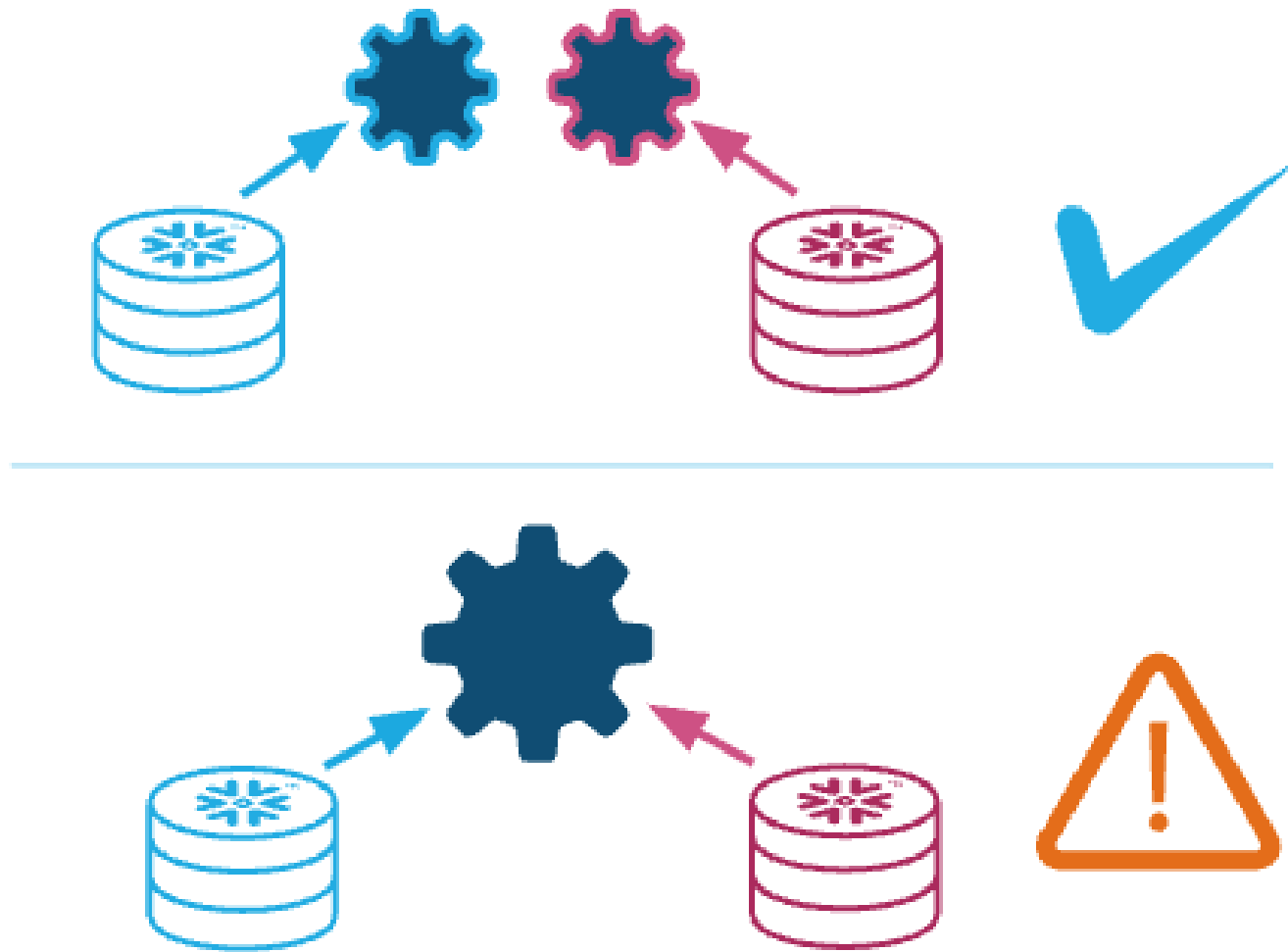
Warehouse cache and Metadata cache

Warehouse cache

- Stores recently scanned micro-partitions on warehouse SSD + memory
- Cleared when the warehouse suspends
- Keep auto-suspend long enough to preserve the cache

Metadata cache

- Always on, lives in Cloud Services
- Stores table statistics
- Some queries answered from metadata



SQL patterns that Prevent Partition Pruning

Prevents pruning:

```
SELECT *  
FROM logistics.shipments  
WHERE MONTH(dispatch_date) = 3;
```

Allows pruning:

```
SELECT *  
FROM logistics.shipments  
WHERE dispatch_date >= '2024-03-01'  
AND dispatch_date < '2024-04-01';
```


Two more SQL patterns to avoid

Avoid SELECT *

-- Reads every column

```
SELECT *  
FROM logistics.shipments  
WHERE region = 'EMEA';
```

-- Reads only what's needed

```
SELECT shipment_id  
, delivery_days  
FROM logistics.shipments  
WHERE region = 'EMEA';
```

Avoid implicit cross-joins

-- Missing join condition = Cartesian

```
SELECT s.shipment_id  
, c.name  
FROM shipments s, carriers c;
```

-- Explicit JOIN with ON condition

```
SELECT s.shipment_id  
, c.name  
FROM shipments s  
JOIN carriers c ON s.carrier_id = c.id;
```


Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

Congratulations!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

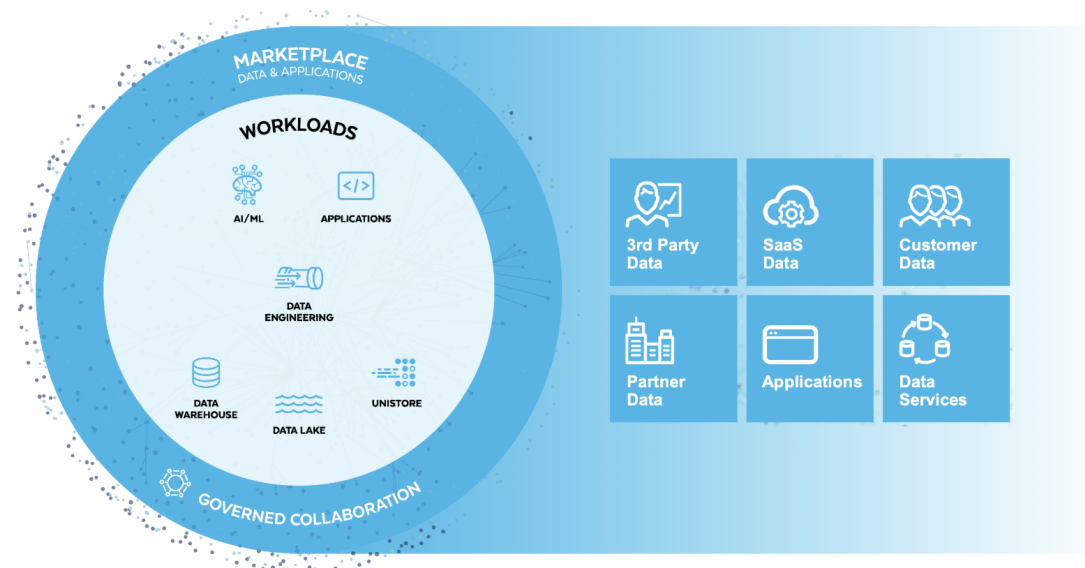


Emily Melhuish

Technical Curriculum Developer,
Snowflake

What You've Covered

- **Chapter 1** — Stages, COPY INTO, Snowpipe, Snowpipe Streaming, data unloading
- **Chapter 2** — Streams, tasks, DAG orchestration, dynamic tables, external and Iceberg tables
- **Chapter 3** — Semi-structured data, aggregate functions, window functions, UDFs, stored procedures
- **Chapter 4** — Query profiling, optimization tools, and caching



The DataCamp Snowflake Track



Final part of a three-course track with DataCamp and Snowflake:

- **Snowflake Architecture** — virtual warehouses, storage, security, sharing
- **Snowflake Management, Governance & Collaboration** — RBAC, data governance, cost control
- **Data Pipeline Automation in Snowflake** — ingestion, orchestration, advanced SQL, performance

Get Ready for the SnowPro Core Certification

You now have everything it takes to sit the **SnowPro Core Certification (COF-C03)**

- One of the most recognised credentials in the data analytics industry
- Covers architecture, security, performance, pipelines, and governance
- Get a **\$50 discount code** within 3 days of completing the track



Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE